

RAPPORT TECHNIQUE

# Étude et choix d'un outil de gestion des cas de test

*Application au projet PFETracker*

*Projet : PFETracker — Plateforme de suivi des Projets de Fin d'Études*

*Réalisé par : Nour Fatnassi & Fatma Hassine*

*Établissement : ENICarthage*

*Filière / Niveau : Génie informatique, 3<sup>ème</sup> année*

# Table des matières

---

|                                                                           |           |
|---------------------------------------------------------------------------|-----------|
| <b>1. Introduction et description du projet .....</b>                     | <b>4</b>  |
| 1.1 Contexte général .....                                                | 4         |
| 1.2 Architecture technique .....                                          | 4         |
| 1.3 Organisation fonctionnelle en modules                                 |           |
| 1.4 Problématique adressée                                                |           |
| 1.5 Besoins fonctionnels par module                                       |           |
| 1.6 Acteurs concernés                                                     | 4         |
| <b>2. Étude des outils de gestion des tests.....</b>                      | <b>6</b>  |
| 2.1 Démarche d'évaluation .....                                           | 6         |
| 2.2 Présentation des outils étudiés .....                                 | 6         |
| <b>3. Étude comparative.....</b>                                          | <b>7</b>  |
| <b>4. Choix de l'outil et justification .....</b>                         | <b>8</b>  |
| 4.1 Outil retenu .....                                                    | 8         |
| 4.2 Justification détaillée.....                                          | 8         |
| 4.3 Pourquoi pas les autres outils .....                                  | 8         |
| 4.4 Synthèse : pourquoi Kiwi TCMS l'emporte face à chaque concurrent..... | 9         |
| <b>5. Installation de l'outil retenu (Kiwi TCMS).....</b>                 | <b>10</b> |
| 5.1 Prérequis .....                                                       | 10        |
| 5.2 Récupération des fichiers officiels .....                             | 10        |
| 5.3 Lancement des conteneurs .....                                        | 10        |
| 5.4 Première connexion .....                                              | 10        |
| <b>6. Intégration des exigences du projet dans l'outil .....</b>          | <b>12</b> |
| 6.1 Création du produit et des versions .....                             | 12        |
| 6.2 Catalogue complet des exigences fonctionnelles.....                   | 12        |
| 6.3 Déclinaison des exigences en cas de test (exemples) .....             | 16        |
| 6.4 Création d'un plan de test et exécution                               |           |
| 6.5 Guide pas-à-pas : introduire une exigence dans Kiwi TCMS              | 17        |

## 6.5 Guide pas-à-pas : introduire une exigence dans Kiwi TCMS

Cette section détaille, étape par étape, comment transformer une exigence du catalogue (§6.2) en cas de test exploitable dans Kiwi TCMS, de la rédaction jusqu'au suivi de son exécution.

### Étape 1 — Retrouver le produit et la classification

Ouvrir le produit PFETTracker (créé en §6.1) et sélectionner la classification correspondant au module de l'exigence à traiter (Module 1, 2 ou 3).

## Étape 2 — Créer le cas de test

Menu TestCases puis Nouveau cas de test. Renseigner : Summary (reformulation de l'exigence en action de vérification), Classification (module concerné), Steps (étapes précises à exécuter), Expected Result (résultat attendu pour chaque étape), et dans le champ Notes la référence de l'exigence du catalogue (ex. REQ-14) pour garder la traçabilité.

## Étape 3 — Regrouper les cas de test dans un plan de test

Menu TestPlans puis Nouveau plan, un plan par module. Y rattacher tous les cas de test créés à l'étape précédente pour ce module.

## Étape 4 — Exécuter le plan (Test Run)

Associer le plan à une version du produit et lancer une exécution. Chaque cas de test est marqué Passé, Échoué ou Bloqué au fur et à mesure de la vérification manuelle.

## Étape 5 — Suivre la couverture

Le tableau de bord du plan de test affiche le pourcentage d'exigences couvertes et leur statut, utilisable comme preuve de conformité au cahier des charges.

*[ Capture d'écran à insérer — création d'un cas de test avec ses étapes ]*

*Figure 6 — Fiche de création d'un cas de test dans Kiwi TCMS*

*[ Capture d'écran à insérer — vue d'ensemble d'un Test Run en cours d'exécution ]*

*Figure 7 — Suivi d'un Test Run et de la couverture des exigences*

## 7. Conclusion..... 18

# 1. Introduction et description du projet

## 1.1 Contexte général

PFETracker est une plateforme web de gestion et de suivi de bout en bout des Projets de Fin d'Études (PFE). Elle permet de coordonner l'ensemble des acteurs impliqués — étudiants, encadrants, responsables de département, directeur des stages et service des stages — autour d'un même espace de travail, depuis l'attribution du sujet jusqu'au suivi détaillé de l'avancement (sprints, tâches, jalons, livrables, réunions et communication).

## 1.2 Architecture technique

Le système repose sur une architecture client-serveur classique :

**Backend** : Java 17 / Spring Boot 3.2, avec Spring Security et authentification par jetons JWT, Spring Data JPA pour la persistance, et Spring WebSocket (STOMP) pour la messagerie et les notifications temps réel.

**Base de données** : MySQL.

**Frontend** : React 18 avec TypeScript, construit avec Vite, interface basée sur des composants Radix UI / MUI.

**Intégrations externes** : génération de liens de réunion via l'API Google Calendar, export de rapports au format Excel (Apache POI) et PDF (OpenPDF).

## 1.3 Organisation fonctionnelle en modules

Le projet est structuré en trois modules fonctionnels, ce qui a une incidence directe sur la manière dont les cas de test doivent être organisés dans l'outil de gestion des tests :

| Module                                         | Responsabilité                                                                                     | Fonctionnalités principales                                                                                                                      |
|------------------------------------------------|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Module 1 – Administration</b>               | Gestion des utilisateurs, des rôles, des invitations et de la sécurité d'accès                     | Création et validation des comptes, gestion des rôles et des départements, affectation encadrant-étudiant, journalisation des actions sensibles  |
| <b>Module 2 – Suivi du PFE</b>                 | Suivi opérationnel du projet : sprints, tâches, jalons, livrables, import de sujets, assistance IA | Création et suivi du PFE, jalons, tâches, livrables, sprints, calcul automatique de la progression, détection intelligente des retards           |
| <b>Module 3 – Communication &amp; pilotage</b> | Messagerie, réunions, commentaires, notifications et tableaux de bord décisionnels                 | Messagerie, commentaires, notifications configurables, planification de réunions avec génération de liens Google Meet, tableaux de bord par rôle |

**Pourquoi ce contexte importe pour le choix de l'outil :** un système multi-rôles organisé en trois modules distincts, avec des règles de sécurité et d'accès propres à chacun, exige une gestion des cas de test structurée (par produit, par version, par catégorie) et une traçabilité claire entre les exigences du cahier des charges et les cas de test associés. C'est le critère central de l'étude qui suit.

## 1.4 Problématique adressée

Avant PFETracker, le suivi d'un Projet de Fin d'Études reposait sur des outils disparates : échanges par email, messageries instantanées, documents partagés, sans espace centralisé. Cela génère plusieurs difficultés concrètes : absence de traçabilité des échanges et des décisions, impossibilité pour un encadrant de suivre plusieurs étudiants sur une vue unique, absence d'outil de supervision pour l'administration à l'échelle d'un département, impossibilité pour le directeur des stages de comparer les départements entre eux, et aucun mécanisme automatique pour détecter un retard ou une inactivité prolongée.

## 1.5 Besoins fonctionnels par module

Module 1 : permettre à chaque acteur (étudiant, encadrant, responsable de département, directeur des stages, service des stages, administrateur) de disposer d'un compte sécurisé, avec une validation contrôlée par le responsable de département, une isolation stricte des données par département, et une vue adaptée à son rôle.

Module 2 : structurer le travail de l'étudiant en jalons, tâches et sprints avec un objectif clair, permettre le dépôt de livrables versionnés, calculer automatiquement une progression fiable, et alerter automatiquement en cas de retard ou d'inactivité prolongée.

Module 3 : permettre une communication directe et tracée entre étudiant et encadrant, la planification de réunions avec génération automatique d'un lien de visioconférence, un système de notifications configurable, et un tableau de bord adapté à chaque rôle pour piloter l'avancement.

## 1.6 Acteurs concernés

Six catégories d'acteurs interagissent avec le système : l'étudiant, l'encadrant, le responsable de département, le directeur des stages, le service des stages, et l'administrateur, chacun avec un périmètre d'accès distinct. Le détail complet des rôles et de leurs permissions est fourni dans le document dédié « Cahier des charges - PFETracker », section 2.

## 2. Étude des outils de gestion des tests

---

### 2.1 Démarche d'évaluation

Huit outils de gestion de cas de test (Test Case Management — TCM) ont été étudiés. L'évaluation s'appuie sur six critères : le coût, le mode d'hébergement, l'existence d'une dépendance à un outil tiers, la qualité de l'API pour l'automatisation, le niveau de maintenance actuel, et le public cible de l'outil.

### 2.2 Présentation des outils étudiés

**TestLink** — le plus ancien outil open-source de gestion de tests (PHP/MySQL). Conçu à l'origine comme alternative gratuite aux outils commerciaux : cas de test, plans, campagnes, suivi des exécutions.

**Kiwi TCMS** — outil open-source moderne (Python/Django), né pour succéder à TestLink avec une architecture actuelle, une API REST/RPC documentée, et une intégration native aux pipelines CI/CD (import de résultats JUnit/pytest).

**Squash TM** — outil open-source français (Henix, Java), axé sur la traçabilité exigences ↔ cas de test ↔ exécutions, très utilisé en entreprise et dans l'enseignement en France.

**Zephyr Scale** — extension de gestion de tests pour Jira, destinée aux équipes déjà équipées de Jira.

**Xray** — concurrent direct de Zephyr, également sous forme de plugin Jira, avec une intégration poussée aux tickets et à la CI.

**TestRail** — outil commercial (Gurock/Idera), SaaS ou on-premise, orienté équipes QA professionnelles avec reporting avancé prêt à l'emploi.

**qTest** — suite commerciale (Tricentis) pour grandes entreprises : gestion de tests, gestion d'exigences et intégration CI à grande échelle.

**PractiTest** — plateforme SaaS commerciale de gestion de tests de bout en bout, destinée aux équipes QA d'entreprise.

### 3. Étude comparative

Le tableau ci-dessous synthétise la comparaison des huit outils selon les six critères retenus.

| Critère                                             | TestLink                                                             | Kiwi TCMS                                                                     | Squash TM                                                             | Zephyr / Xray                                   | TestRail                                           | qTest / PractiTest   |
|-----------------------------------------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------|-------------------------------------------------|----------------------------------------------------|----------------------|
| <b>Coût</b>                                         | Gratuit                                                              | Gratuit                                                                       | Gratuit                                                               | Payant                                          | Payant (essai gratuit)                             | Payant, élevé        |
| <b>Hébergement</b>                                  | Auto-hébergé (PHP)                                                   | Auto-hébergé (Docker)                                                         | Auto-hébergé (Java)                                                   | Nécessite Jira                                  | SaaS / on-prem payant                              | SaaS                 |
| <b>Dépendance externe</b>                           | Aucune                                                               | Aucune                                                                        | Aucune                                                                | Jira obligatoire                                | Aucune (mais payant)                               | Aucune (mais payant) |
| <b>API / automatisation</b>                         | Faible                                                               | Bonne (REST/RPC)                                                              | Correcte (module dédié requis)                                        | Bonne (via Jira)                                | Bonne                                              | Bonne                |
| <b>Maintenance (2026)</b>                           | Quasi à l'arrêt                                                      | Active                                                                        | Active                                                                | Active (commercial)                             | Active (commercial)                                | Active (commercial)  |
| <b>Public cible</b>                                 | Historique / académique                                              | Équipes techniques, CI/CD                                                     | Grandes entreprises, exigences complexes                              | Équipes déjà sous Jira                          | PME / équipes QA                                   | Grandes entreprises  |
| <b>Compatibilité versions / dépendances système</b> | Fragile — nécessite PHP/MySQL anciens, mauvaise compatibilité PHP 8+ | Aucun souci — conteneurisé (Docker), versions figées et testées par l'éditeur | Correcte — JDK/Tomcat requis, mais image Docker officielle disponible | Gérée par Atlassian (liée à la version de Jira) | Aucun souci (SaaS) / gérée par l'éditeur (on-prem) | Aucun souci (SaaS)   |

**Point technique complémentaire — compatibilité des versions :** La documentation officielle de TestLink recommande **MySQL 5.5.16** (minimum MySQL 4.x) et un support PHP allant de PHP 5.x à PHP 7.4 — des versions vieilles de plus de dix ans, jamais validées avec MySQL 8, qui a changé son plugin d'authentification par défaut (caching\_sha2\_password) et durci ses modes SQL stricts. Installer TestLink sur un environnement moderne (MySQL 8 par défaut) expose donc à des erreurs de connexion ou de requêtes SQL, sauf à reconfigurer manuellement le serveur en mode de compatibilité (mysql\_native\_password) — un bricolage supplémentaire. Kiwi TCMS ne présente pas ce risque : étant distribué officiellement via Docker, l'ensemble de ses dépendances (Python, Django, base de données) est figé et testé à l'intérieur du conteneur par l'éditeur, indépendamment des versions installées sur la machine hôte.

## 4. Choix de l'outil et justification

---

### 4.1 Outil retenu

**Outil choisi : Kiwi TCMS.** Kiwi TCMS est l'outil de gestion des cas de test retenu pour le projet PFETracker.

### 4.2 Justification détaillée

**Gratuit et open-source** : aucun coût, pas de limite d'essai susceptible d'expirer avant la fin du projet.

**Aucune dépendance externe** : contrairement à Zephyr/Xray, aucun outil tiers (Jira) n'est requis.

**Léger à déployer** : installation en une commande via Docker Compose, contrairement à Squash TM (serveur Java/Tomcat à configurer) ou aux suites SaaS d'entreprise.

**Activement maintenu** : contrairement à TestLink, dont le développement est quasiment à l'arrêt.

**API REST/RPC scriptable** : permet de relier directement les futurs tests automatisés (JUnit côté backend) à l'outil, sans module supplémentaire lourd — contrairement à Squash TM qui nécessite un module séparé (Squash AutoM8) pour ce lien.

### 4.3 Pourquoi pas les autres outils

**Zephyr Scale et Xray** — outils très complets, mais tous deux sont des extensions Jira. Sans instance Jira déjà en place dans le projet, il faudrait d'abord déployer et financer Jira, puis ajouter le plugin (lui-même payant au-delà d'un petit nombre d'utilisateurs) : coût et dépendance en cascade injustifiables pour un projet académique.

**TestRail** — outil solide et répandu en entreprise, mais SaaS payant dès la fin de la période d'essai (30 jours) : la solution ne survivrait pas à la durée du projet.

**qTest / PractiTest** — suites « entreprise » avec tarification par utilisateur, conçues pour des équipes QA de plusieurs dizaines de personnes avec des workflows d'approbation complexes : disproportionnées pour une petite équipe projet.

**TestLink** — référence historique, mais développement quasiment à l'arrêt, interface datée, et nécessite une pile PHP/MySQL distincte de la pile Java/React du projet, sans bénéfice technique en contrepartie. À cela s'ajoute un problème concret de **compatibilité de versions**: conçu pour d'anciennes versions de PHP, son support de PHP 8+ est incertain, ce qui imposerait de maintenir une version PHP obsolète spécifiquement pour cet outil — un risque que Kiwi TCMS élimine grâce à sa distribution Docker officielle, où toutes les versions sont figées dans le conteneur.

**Squash TM** — concurrent sérieux, très bon sur la traçabilité exigences/tests, mais plus lourd à installer (serveur d'application Java) et dont le lien avec des tests automatisés passe par un

module supplémentaire (Squash AutoM8), plus complexe à mettre en œuvre qu'un simple appel API pour un projet de cette taille.

## 4.4 Synthèse : pourquoi Kiwi TCMS l'emporte face à chaque concurrent

| Concurrent                 | Sa force                                           | Sa limite pour ce projet                                                                                      | Pourquoi Kiwi TCMS l'emporte ici                                                           |
|----------------------------|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>TestLink</b>            | Historique, connu dans le milieu académique        | Stack PHP/MySQL obsolète (MySQL 5.5.16 recommandé), incompatible avec un environnement moderne sans bricolage | Aucune contrainte de version : tout est figé et testé dans le conteneur Docker officiel    |
| <b>Squash TM</b>           | Meilleure traçabilité exigences ↔ tests du marché  | Serveur Java/Tomcat à configurer ; lien avec les tests automatisés via un module séparé (Squash AutoM8)       | Installation en une commande ; API REST directement exploitable sans module supplémentaire |
| <b>Zephyr Scale / Xray</b> | Intégration très poussée avec les tickets et la CI | Nécessitent Jira, absent du projet ; coût de licence dès que l'équipe dépasse quelques utilisateurs           | Fonctionne seul, sans aucun outil tiers à acheter ou déployer au préalable                 |
| <b>TestRail</b>            | Reporting prêt à l'emploi, très ergonomique        | Payant dès la fin de l'essai de 30 jours — ne couvre pas la durée d'un PFE                                    | Gratuit indéfiniment, sans date d'expiration                                               |
| <b>qTest / PractiTest</b>  | Très complet pour de grandes équipes QA            | Tarification par utilisateur/mois, workflows conçus pour des dizaines de testeurs                             | Dimensionné pour une petite équipe projet, sans surcoût ni complexité inutile              |

## 5. Installation de l'outil retenu (Kiwi TCMS)

### 5.1 Prérequis

- Docker Desktop installé et fonctionnel (Windows / Linux / macOS).
- Docker Compose (intégré à Docker Desktop).
- Un accès réseau sortant pour télécharger les images officielles.

### 5.2 Récupération des fichiers officiels

```
git clone https://github.com/kiwitcms/Kiwi.git  
cd Kiwi
```

### 5.3 Lancement des conteneurs

```
docker-compose up -d
```

Cette commande démarre les deux conteneurs nécessaires : le conteneur applicatif Kiwi TCMS et le conteneur de base de données (MariaDB).

### 5.4 Première connexion

L'interface web est accessible via un navigateur à l'adresse `http://localhost/` (ou le port configuré dans le fichier `docker-compose.yml`). La connexion se fait avec les identifiants créés à l'étape précédente.

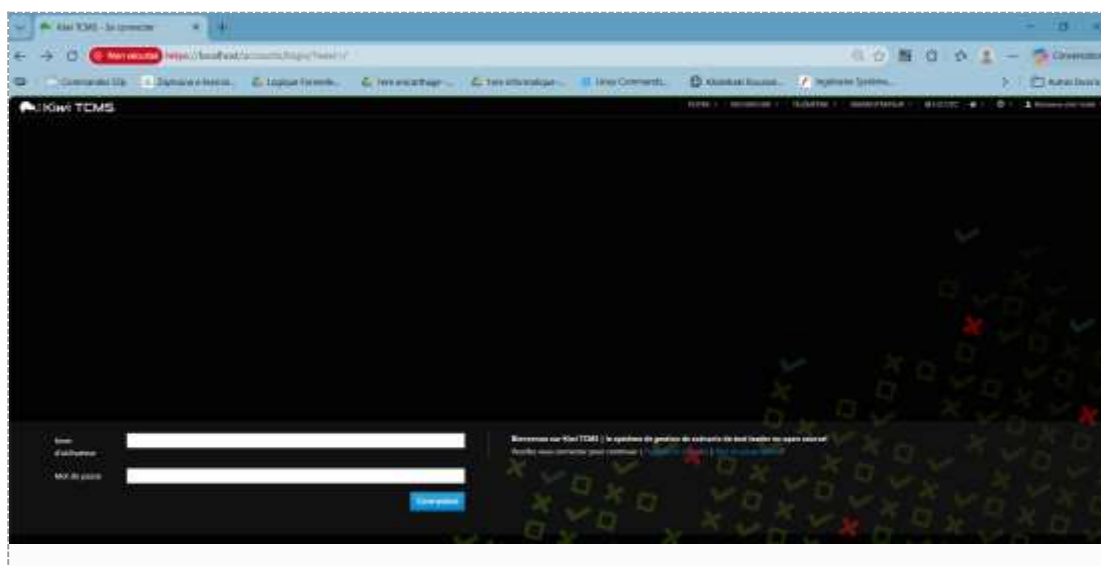


Figure 1 — Page de connexion de Kiwi TCMS



## 6. Intégration des exigences du projet dans l'outil

---

### 6.1 Création du produit et des versions

Dans le menu Administration → Products, créer un produit nommé PFETracker, puis définir une version par jalon du projet (par exemple v1.0). Créer ensuite une Classification par module fonctionnel afin de refléter l'organisation réelle du système :

- Module 1 - Administration
- Module 2 - Suivi du PFE
- Module 3 - Communication & pilotage

[ Capture d'écran à insérer — création du produit PFETracker ]

Figure 3 — Création du produit et de ses classifications

### 6.2 Catalogue complet des exigences fonctionnelles

Ce catalogue recense **72 exigences** issues du cahier des charges mis à jour du projet (document dédié « Cahier des charges - PFETracker », version 2.0), vérifiées une à une par rapport à l'implémentation réelle, organisées par module puis par sous-thème fonctionnel. C'est ce catalogue, et non quelques exemples isolés, qui doit être intégralement retranscrit en cas de test dans Kiwi TCMS pour garantir une couverture complète.

#### Module 1 — Administration

##### 1.1 Authentification et sécurité

1. Le système doit permettre à un utilisateur de s'inscrire
2. Le système doit permettre à un utilisateur de se connecter et de recevoir un jeton JWT
3. Le système doit permettre de rafraîchir un jeton JWT expiré via un refresh token, avec rotation de ce dernier à chaque rafraîchissement
4. Le système doit permettre à un utilisateur connecté de se déconnecter en révoquant son refresh token
5. Le système doit permettre de demander puis d'effectuer une réinitialisation de mot de passe par email
6. Le système doit permettre à un utilisateur connecté de changer son mot de passe
7. Le système doit limiter le nombre de tentatives de connexion (10/min) et d'inscription (5/min) par IP pour prévenir la force brute

- 8.**Un compte étudiant ou encadrant nouvellement créé doit rester inactif jusqu'à validation explicite du chef de département

### **1.2 Gestion des comptes utilisateurs**

- 9.**Le système doit permettre à un utilisateur de consulter et mettre à jour son propre profil
- 10.**Le système doit permettre à un chef de département ou un directeur de consulter le profil d'un utilisateur par ID
- 11.**Le système doit permettre à un administrateur de créer, supprimer, activer/désactiver, déverrouiller, réinitialiser le mot de passe et changer le rôle d'un compte
- 12.**Le système doit permettre à un administrateur de lister tous les comptes, filtrables par rôle, ou les comptes verrouillés

### **1.3 Gestion des départements**

- 13.**Le système doit permettre à un administrateur de créer, modifier, supprimer et lister les départements, et d'y assigner un chef
- 14.**Le système doit isoler strictement les données entre départements : un chef ne peut agir que sur son propre département, même si l'ID est manipulé dans l'URL
- 15.**Le système doit permettre à un chef de département de consulter et valider/refuser (avec motif) les comptes en attente de son département
- 16.**Le système doit permettre à un chef de département de gérer les comptes de son département (activation, déverrouillage, réinitialisation mot de passe, changement de département)
- 17.**Le système doit permettre à un chef de département de consulter la charge de travail de chaque encadrant et de configurer une limite maximale d'étudiants par encadrant
- 18.**Le système doit permettre à un chef de département de régénérer le code d'identification d'un encadrant

### **1.4 Affectation étudiant–encadrant**

- 19.**Le système doit permettre d'affecter, forcer l'affectation, réaffecter ou rompre l'affectation entre un étudiant et un encadrant
- 20.**Le système doit permettre à un directeur des stages de transférer un étudiant vers un autre département
- 21.**Le système doit déclencher une vérification et une affectation automatique des étudiants non affectés après un délai de 21 jours
- 22.**Le système doit permettre d'importer en masse les affectations encadrant–étudiant depuis un fichier Excel/CSV
- 23.**Le système doit identifier les étudiants non affectés ou « orphelins »

### **1.5 Suivi des stages**

- 24.**Le système doit permettre de lister les étudiants en stage, ceux en alerte, et les encadrants disponibles
- 25.**Le système doit signaler les stages proches de l'expiration et les stages expirés
- 26.**Le système doit fournir un tableau de bord dédié au service des stages, au directeur et à l'administrateur

## 1.6 Export de rapports

- 27. Le système doit permettre au directeur des stages d'exporter un rapport global au format Excel et PDF (§4.4.3)
- 28. Le système doit permettre à un chef de département d'exporter le rapport de son département au format Excel et PDF (§4.3.3)

## 1.7 Journalisation, audit et système

- 29. Le système doit permettre à un chef de département de consulter les journaux d'audit des utilisateurs de son département
- 30. Le système doit permettre à un administrateur de consulter l'ensemble des journaux d'audit et de réinitialiser le système
- 31. Le traitement de l'envoi d'email doit être asynchrone et ne jamais bloquer la réponse API (cible < 2s) (§7.2)

## 1.8 Notifications

- 32. Le système doit permettre à un utilisateur de consulter ses notifications et de les marquer comme lues, individuellement ou toutes à la fois

*Le mécanisme d'invitation par code (InvitationController) est désactivé dans le code (routes commentées) — non retenu comme exigence active.*

# Module 2 — Suivi du PFE

## 2.1 Gestion des PFE

- 33. Le système doit permettre de créer un PFE, générant automatiquement 6 jalons par défaut
- 34. Le système doit retourner les PFE visibles selon le rôle : ses PFE pour l'étudiant, ceux encadrés pour l'encadrant, tous pour l'administration
- 35. Le système doit permettre de consulter, mettre à jour et valider/refuser la fiche projet d'un PFE
- 36. Le système doit générer un rapport hebdomadaire par encadrant, consultable dans le tableau de bord et envoyé par email chaque lundi (§5.7.2)
- 37. Le système doit permettre d'importer en masse des sujets/affectations de PFE depuis un fichier

## 2.2 Sprints

- 38. Le système doit permettre de créer, lister et consulter les sprints d'un PFE
- 39. Le système doit permettre à l'étudiant de soumettre le bilan de son sprint, et à l'encadrant de le valider ou le refuser
- 40. Le système doit permettre de clôturer un sprint
- 41. Un sprint doit suivre un cycle de statuts : planifié → en cours → bilan soumis → validé → clôturé

## 2.3 Jalons

- 42. Le système doit permettre de créer un ou plusieurs jalons pour un PFE, de les lister, consulter et modifier
- 43. Un jalon doit suivre un statut parmi : non démarré, en cours, terminé

## 2.4 Tâches et sous-tâches

- 44. Le système doit permettre de créer une tâche rattachée à un jalon, de la lister par jalon ou par PFE, et de la consulter

- 45. Le système doit permettre de changer le statut d'une tâche selon le cycle : non démarrée → en cours → soumise → validée / à corriger / annulée
- 46. Le système doit permettre de créer une sous-tâche et de basculer son état fait/non fait
- 47. Le système doit conserver un historique des changements de statut d'une tâche
- 48. Chaque tâche doit pouvoir recevoir une priorité : basse, normale, haute, critique

## 2.5 Livrables

- 49. Le système doit permettre d'uploader et de lister des livrables rattachés à une tâche ou à un jalon

## 2.6 Assistance et détection intelligente (IA)

- 50. Le système doit générer des suggestions (tâches, user stories, spécifications) via un assistant IA génératif, et permettre de les ajouter individuellement ou en bloc comme tâches réelles
- 51. Le système doit détecter automatiquement chaque nuit à 2h00 : tâches en retard, jalons en retard, stagnation étudiant (seuil configurable), PFE à risque, sprint non clôturé (§5.7, §5.7.1)
- 52. Le système doit permettre de consulter les alertes actives d'un PFE, de les résoudre, et de déclencher manuellement la détection

# Module 3 — Communication & pilotage

## 3.1 Messagerie

- 53. Le système doit permettre d'envoyer un message privé dans une conversation liée à un PFE
- 54. Le système doit permettre de consulter une conversation paginée, de la filtrer par plage de dates, et d'y rechercher par mot-clé
- 55. Le système doit permettre de rechercher un mot-clé dans l'ensemble des messages d'un PFE
- 56. Le système doit permettre de consulter les messages non lus, leur nombre, et de les marquer comme lus individuellement ou en lot
- 57. Le système doit archiver automatiquement les messages d'un PFE clôturé/suspendu/soutenu, ainsi que les messages de plus de 90 jours

## 3.2 Réunions et intégration Google Calendar/Meet

- 58. Le système doit permettre de créer une réunion avec génération automatique d'un lien Google Meet (§6.3.1)
- 59. Le système doit permettre d'accepter/refuser une invitation, de modifier ou d'annuler une réunion
- 60. Le système doit permettre de consulter une réunion, les réunions d'un PFE (paginées), ou les réunions à venir de l'utilisateur
- 61. Le système doit permettre d'ajouter un compte-rendu à une réunion
- 62. Le système doit permettre à un utilisateur d'autoriser, révoquer et consulter le statut de l'accès OAuth2 à son Google Calendar (§6.3.1)
- 63. En cas d'échec de génération d'un lien Meet (accès Google non autorisé), le système doit basculer automatiquement sur un lien de secours (§6.3.1)

## 3.3 Commentaires

- 64. Le système doit permettre d'ajouter un commentaire sur une tâche avec support des mentions « @ », de les lister (paginés ou non) et d'en compter le nombre

### 3.4 Notifications

- 65. Le système doit permettre de consulter les notifications paginées, un centre limité aux 30 dernières, et les notifications non lues avec leur compteur
- 66. Le système doit permettre de marquer une, plusieurs ou toutes les notifications comme lues (y compris automatiquement lors du clic sur l'action associée), et de les supprimer
- 67. Le système doit permettre à chaque utilisateur de configurer ses préférences de notification par canal (email, in-app, websocket) et par type d'événement

### 3.5 Tableaux de bord par rôle

- 68. Le système doit fournir un tableau de bord dédié pour chaque rôle : étudiant, encadrant, chef de département, directeur des stages

## Règles transversales de sécurité/accès

- 69. Chaque contrôleur sensible est protégé par une règle de rôle (@PreAuthorize) correspondant à l'un des rôles : ADMIN, DIRECTEUR, CHEF\_DEPARTEMENT, SERVICE\_STAGE, ENCADRANT, ETUDIANT
- 70. Un chef de département ne peut agir que sur les données de son propre département, avec vérification explicite d'appartenance à chaque requête (§4.1.3)
- 71. Les endpoints d'authentification sensibles sont soumis à une limitation de débit par IP (§7.1)
- 72. Le endpoint public de callback Google (/v3/public/google/callback) est volontairement non protégé par JWT, car appelé par une redirection du navigateur initiée par Google

## 6.3 Déclinaison des exigences en cas de test (exemples)

Chaque exigence du catalogue ci-dessus doit être retranscrite en un ou plusieurs cas de test (TestCase) dans Kiwi TCMS, rattachés à la classification du module correspondant. Exemples représentatifs couvrant les trois modules :

| Exigence      | Exigence fonctionnelle                           | Cas de test associé                                                                                                    |
|---------------|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| n°14          | Isolation stricte des données entre départements | Vérifier qu'un chef de département ne peut pas accéder aux données d'un autre département en modifiant l'ID dans l'URL |
| n°27 (§4.4.3) | Export du rapport directeur au format PDF        | Vérifier que l'export génère un fichier PDF lisible contenant les statistiques globales                                |
| n°41          | Cycle de statuts d'un sprint                     | Vérifier qu'un sprint ne peut pas passer directement de "planifié" à "clôturé" sans étapes intermédiaires              |
| n°51          | Détection automatique nocturne des retards       | Vérifier qu'une tâche dépassant son échéance génère une alerte après l'exécution planifiée de 2h00                     |
| n°58 (§6.3.1) | Génération d'un lien de réunion Google Meet réel | Vérifier qu'une réunion créée génère un lien Google Meet valide et fonctionnel                                         |
| n°67          | Préférences de notification par canal            | Vérifier que désactiver le canal email empêche bien l'envoi d'un email pour ce type d'événement                        |

*[ Capture d'écran à insérer — fiche détaillée d'un cas de test ]*

Figure 4 — Fiche détaillée d'un cas de test dans Kiwi TCMS

## 6.4 Création d'un plan de test et exécution

Les cas de test sont regroupés dans un Test Plan par module, puis exécutés via un Test Run associé à une version du produit. Chaque exécution enregistre un statut (Passé, Échoué, Bloqué) et peut être alimentée automatiquement à partir des résultats des tests JUnit via l'API REST de Kiwi TCMS.

*[ Capture d'écran à insérer — exécution d'un plan de test ]*

Figure 5 — Suivi de l'exécution d'un plan de test

## 7. Conclusion

---

L'étude comparée de huit outils de gestion des cas de test a permis d'identifier Kiwi TCMS comme la solution la plus adaptée au contexte du projet PFETracker : gratuite, autonome (sans dépendance à Jira), légère à déployer, activement maintenue, et dotée d'une API permettant de relier directement les tests automatisés aux exigences du cahier des charges. Sa mise en place — détaillée dans ce rapport, de l'installation via Docker jusqu'à la structuration des cas de test par module — fournit une base méthodologique solide pour garantir la traçabilité et la non-régression tout au long du développement du projet.